Graph Algorithms

BFS

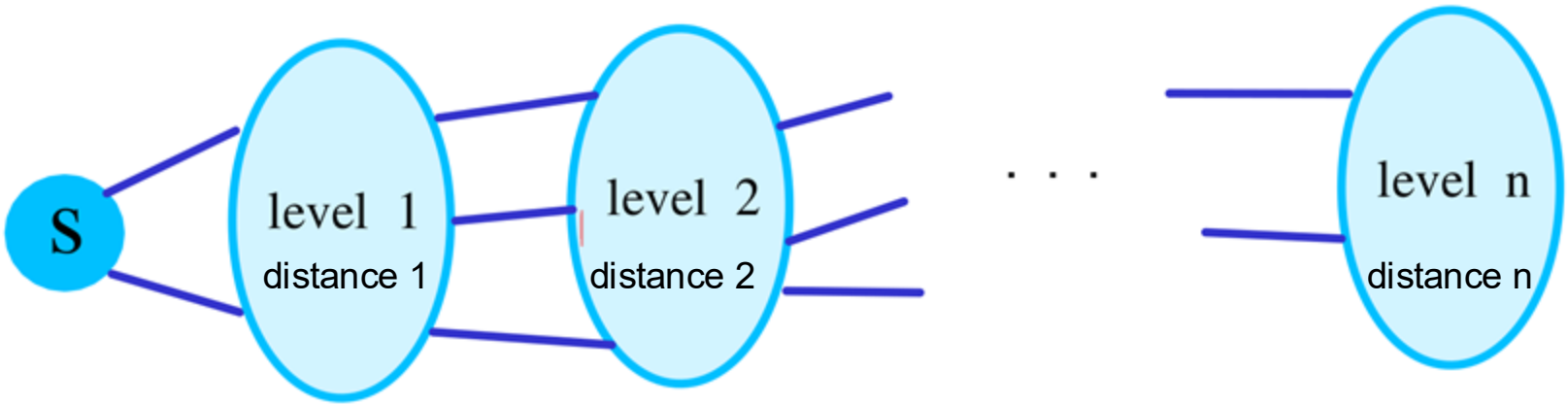
Readings

- http://algorithmics.lsi.upc.edu/docs/Dasgupta-Papadimitriou-Vazirani.pdf
  - Chapter 3
- CLR
  - Chapter 22

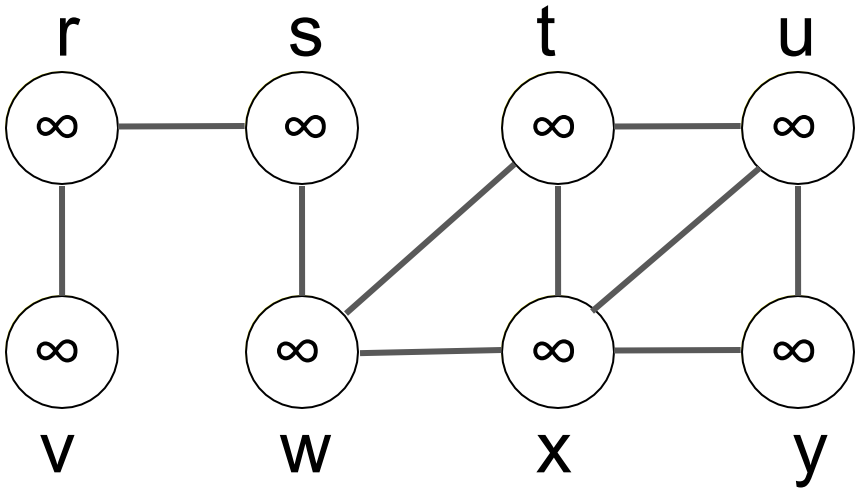
BFS

Breadth-First search

- A graph traversal algorithm
- **Input**: A graph G=(V, E) and a source vertex s
- **Output**:
  - Visits the vertices in order of their distance from s
  - Find the shortest distance from s to each reachable vertex



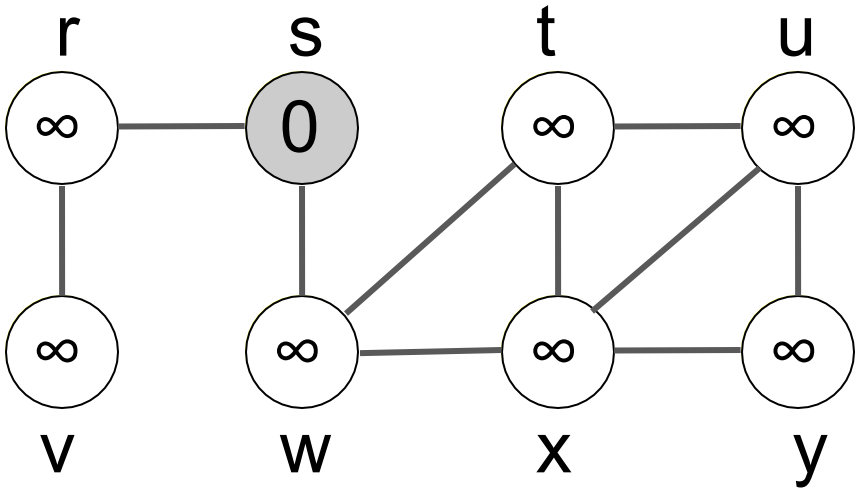
BFS Example



Queue



BFS Example

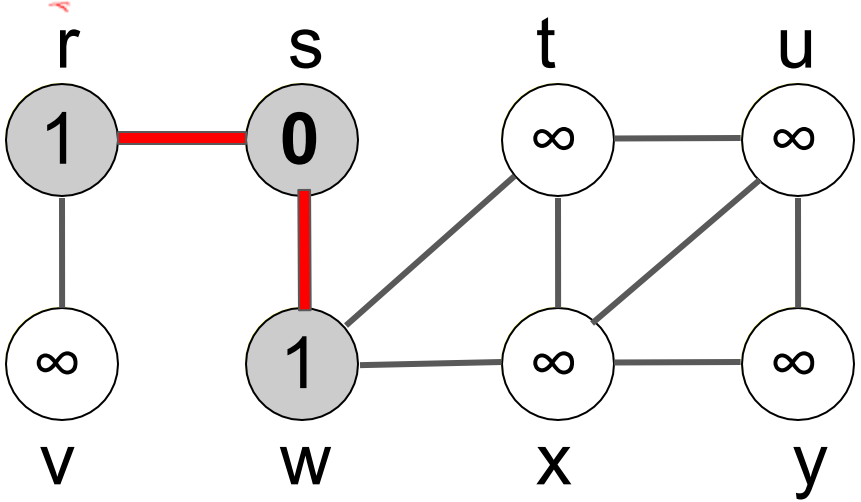


Queue

s

0

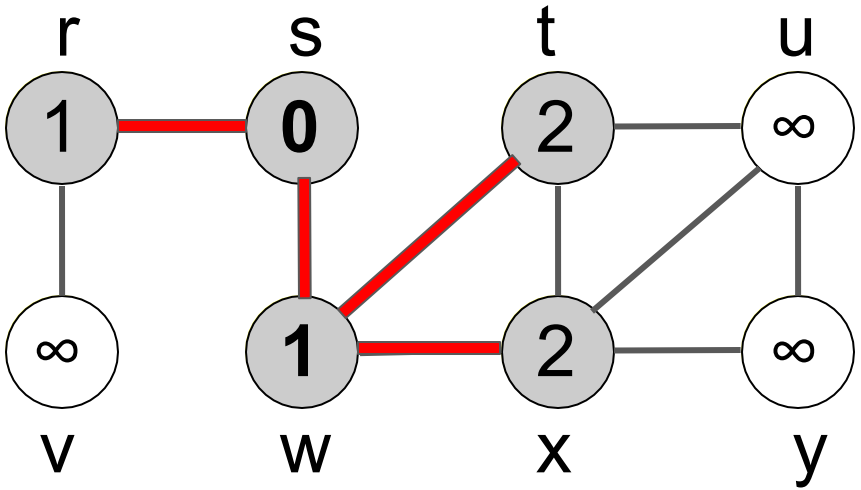
BFS Example



Queue

w	r
1	1

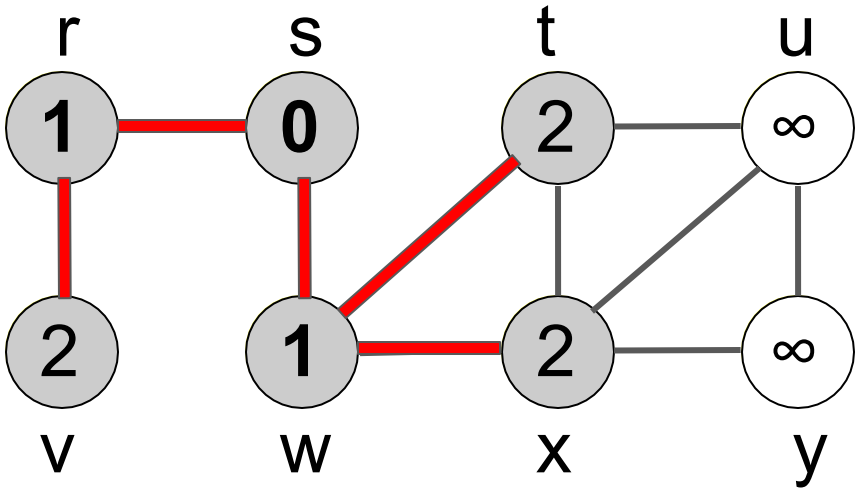
BFS Example



Queue

r	t	x
1	2	2

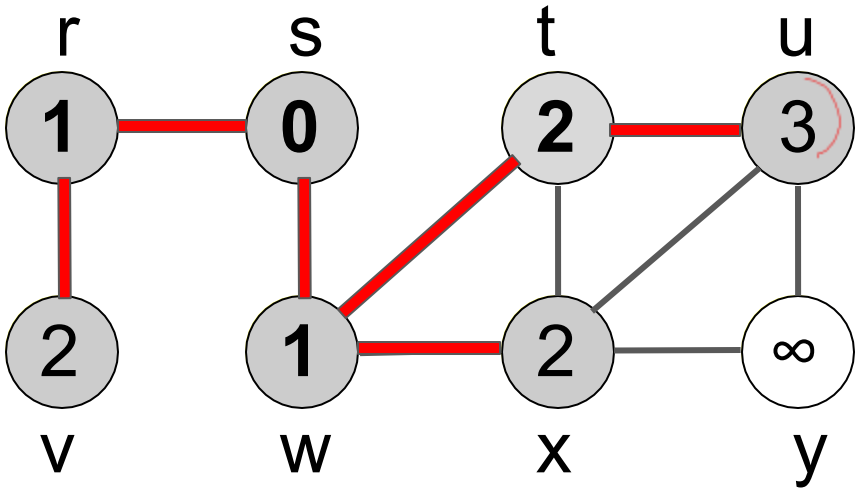
BFS Example



Queue

t	x	v
2	2	2

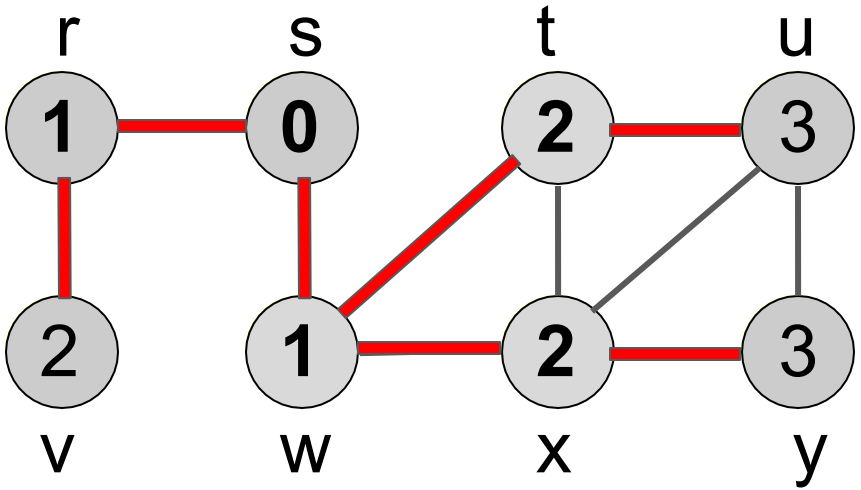
BFS Example



Queue

x	v	u
2	2	3

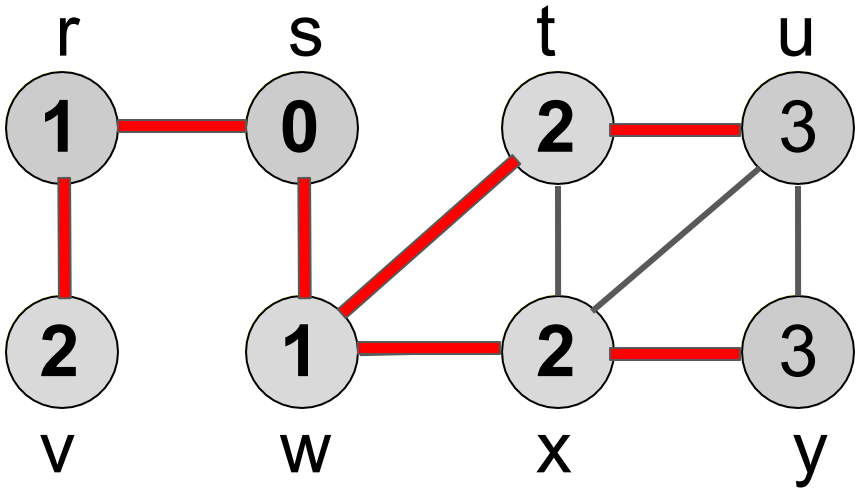
BFS Example



Queue

v	u	y
2	3	3

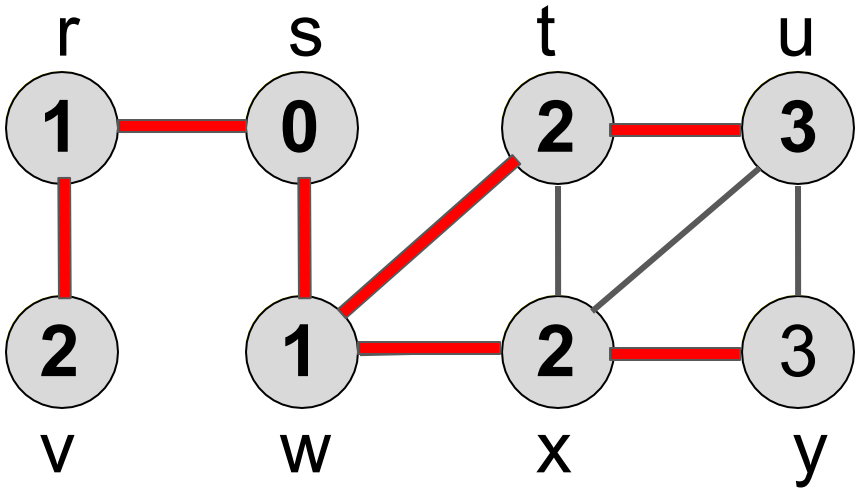
BFS Example



Queue

u	y
3	3

BFS Example

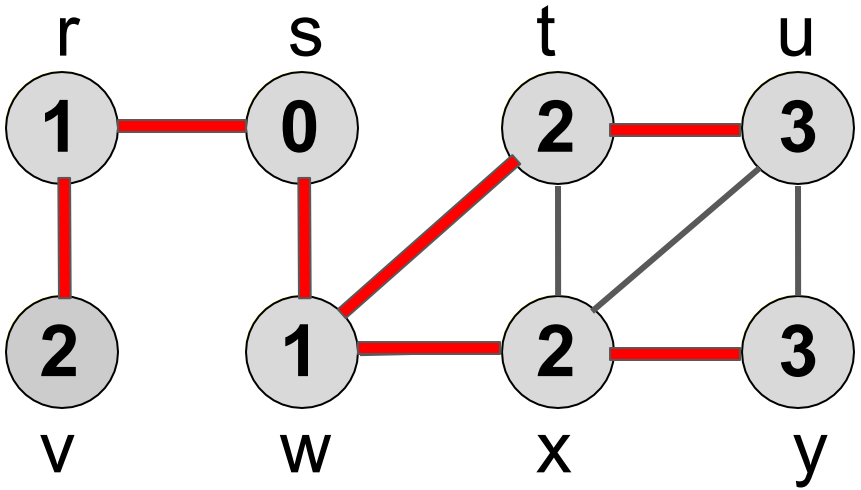


Queue

y

3

BFS Example



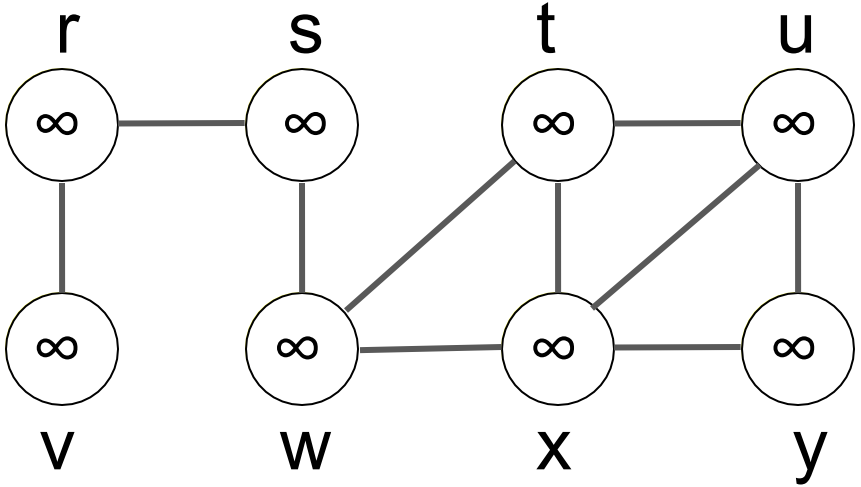
Queue

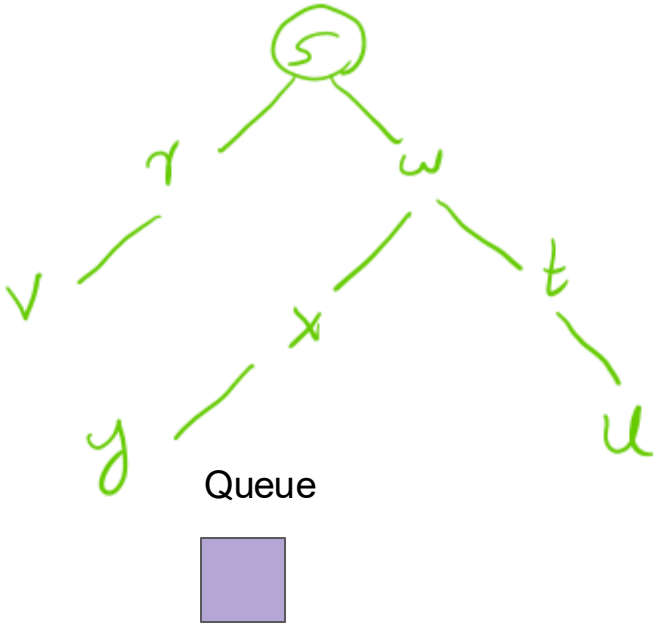
$\varnothing$

Breadth-First search: pseudocode

```
BFS(G, s)
    for each node u in G:
        dist[u] = ∞
    dist[s] = 0
    Q = [s]
    while Q not empty
        u = dequeue(Q)
        for each neighbor v of u:
            if (dist[v] = ∞):
               enqueue(Q, v)
               dist[v] = dist[u] + 1
```

Example





BFS: Runtime Analysis: Naive Analysis

```
BFS(G, s)
    Q = ∅
    for each node u in G:
        dist[u] = ∞
    dist[s] = 0
    Q = [s]
    while Q not empty
        u = dequeue(Q)
        for each neighbor v of u:
            if (dist[v] = ∞):
               enqueue(Q, v)
               dist[v] = dist[u] + 1
```

O(V): for every vertex in G

O(V)

Runtime: $O(V^2)$

BFS: Runtime Analysis: Aggregate Analysis

```
BFS(G, s)
    Q = ∅
    for each node u in G:
        dist[u] = ∞
    dist[s] = 0
    Q = [s]
    while Q not empty
        u = dequeue(Q)
        for each neighbor v of u:
            if (dist[v] = ∞):
               enqueue(Q, v)
               dist[v] = dist[u] + 1
```

O(V): for every vertex in G

O(deg(u))

$n = |V|$

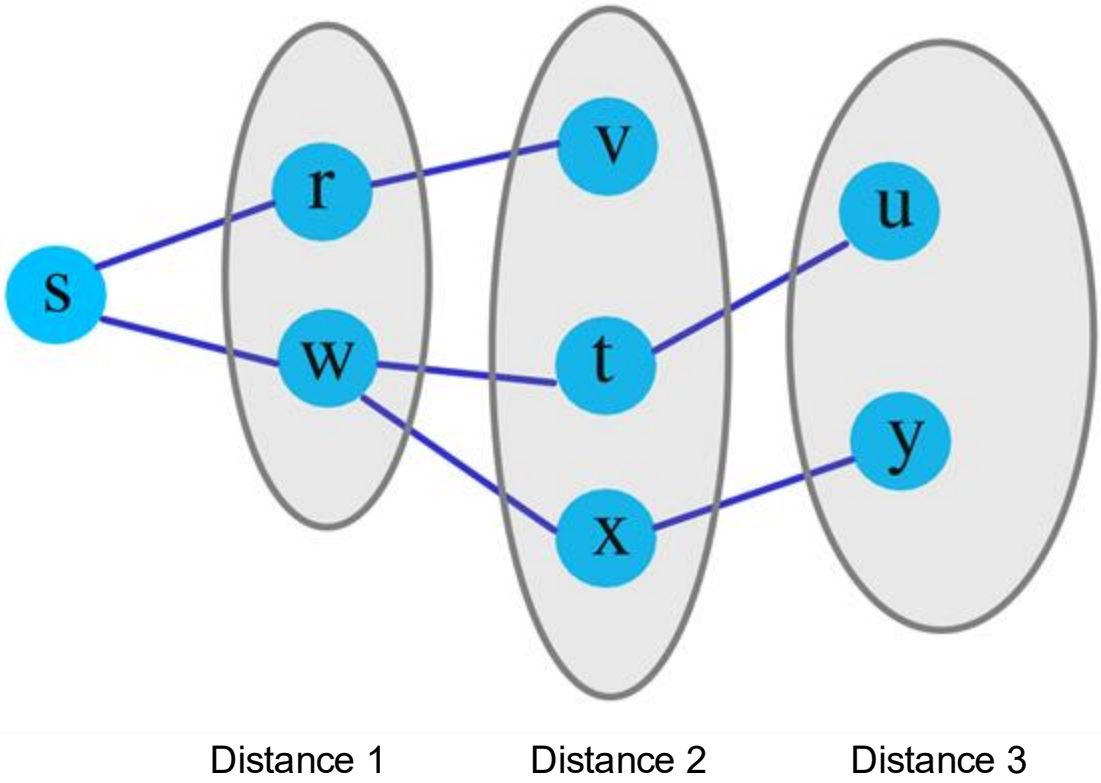
$m = |E|$

- Each vertex is enqueued at most once (when it is discovered)
- When a vertex u is dequeued, the for loop is executed for **deg(u)** iterations

So the total time complexity is $O(n + \sum_{u \in V} deg(u))$ = O(n+m)

$= 2E$

BFS Example

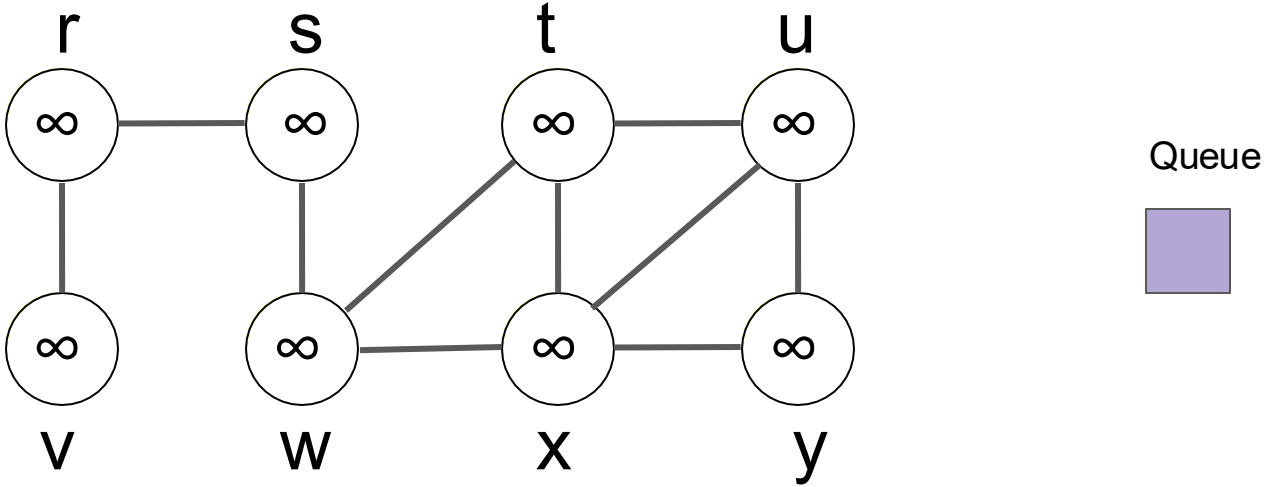


BFS: Finding the path from s to v

- **How to trace back a path from s to v**
  - We can add an array **parent[v]**
  - When a vertex v is discovered within the for loop of vertex u, then we set parent[v] = u.
  - Now to trace out a path v to s, we just need to write a for loop that starts from v and keep going to its parent until we reach vertex s
  - For all vertices v reachable from s, the edges (v, parent[v]) form a tree, called the **BFS tree**
- We store level (distance) in a separate array

BFS Example

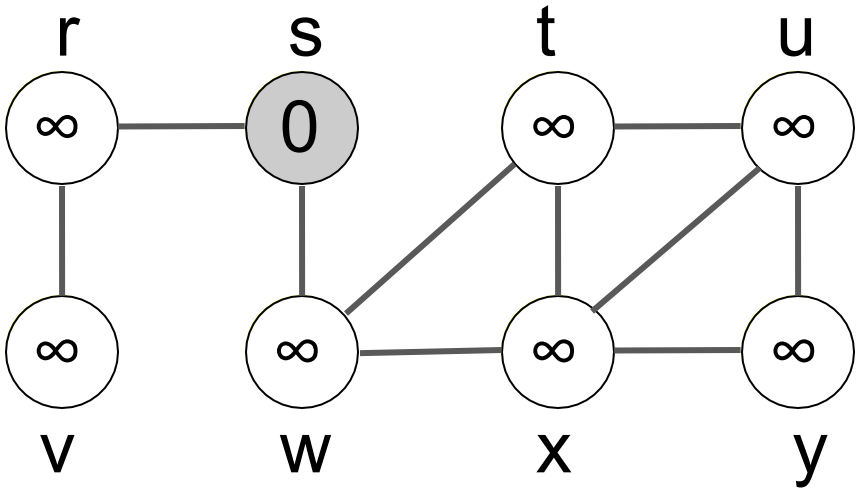




BFS Example

parents:

r	s	v	w	t	x	u	y
Null	Null	Null	Null	Null	Null	Null	Null



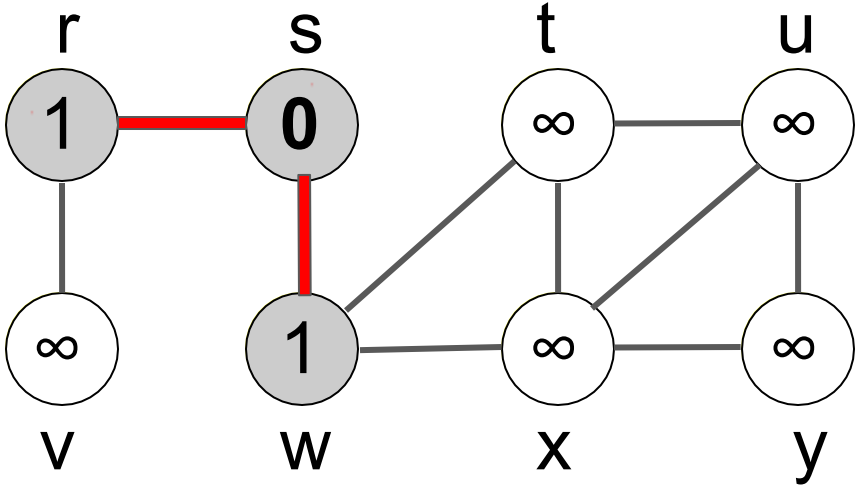
Queue

s

0

BFS Example





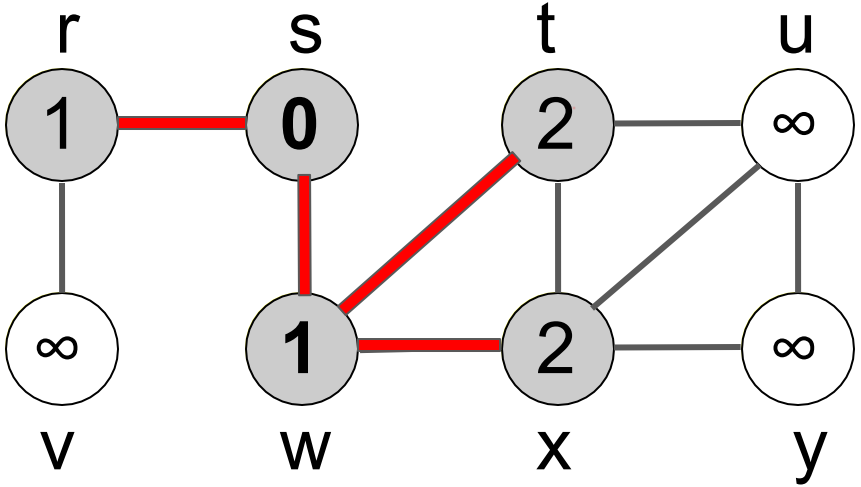
Queue

w	r
1	1

BFS Example

parents:

r	s	v	w	t	x	u	y
s			s	w	w		

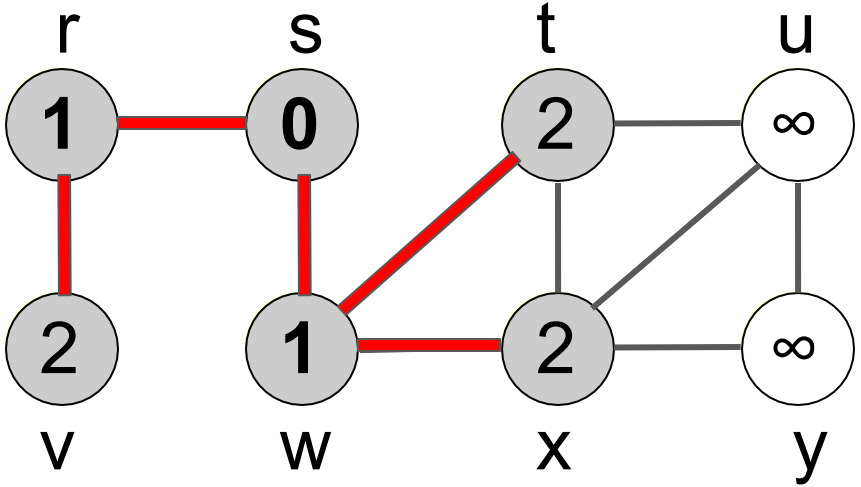


Queue

r	t	x
1	2	2

BFS Example





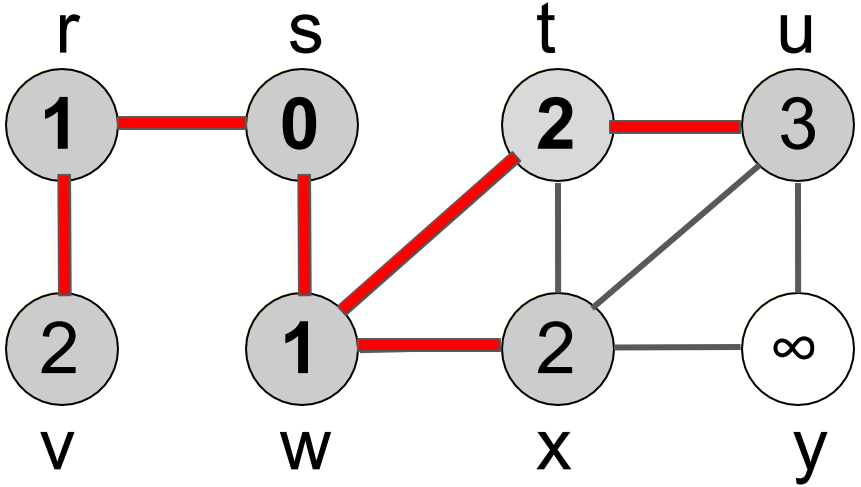
Queue

t	x	v
2	2	2

BFS Example

parents:

r	s	v	w	t	x	u	y
s		r	s	w	w	t	



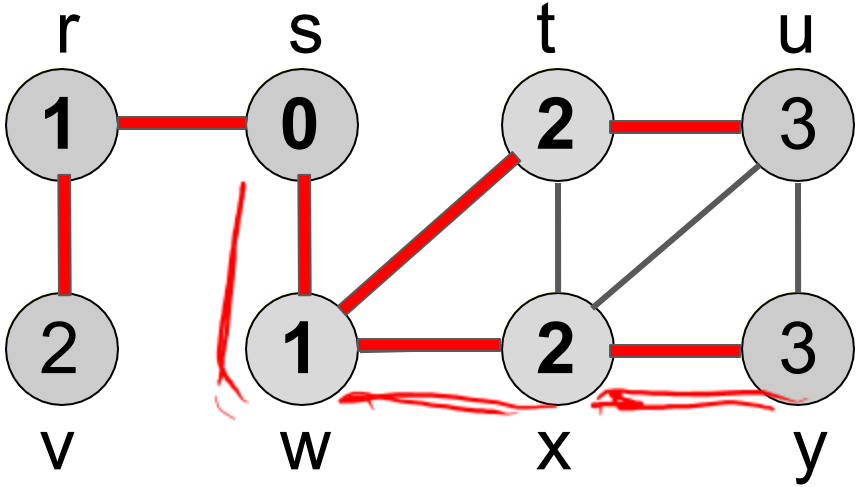
Queue

x	v	u
2	2	3

BFS Example

parents:

r	s	v	w	t	x	u	y
s	Null	r	s	w	w	t	x

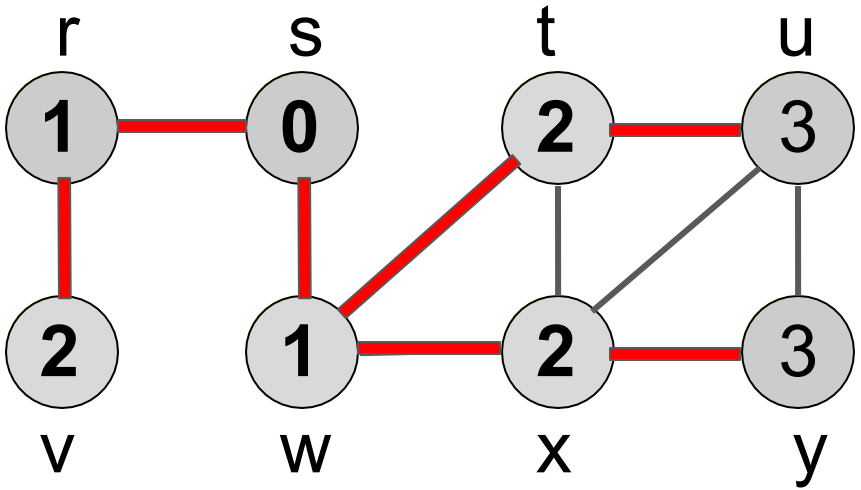


Queue

v	u	y
2	3	3

BFS Example

	r	s	v	w	t	x	u	y
parents:	s	Null	r	s	w	w	t	x

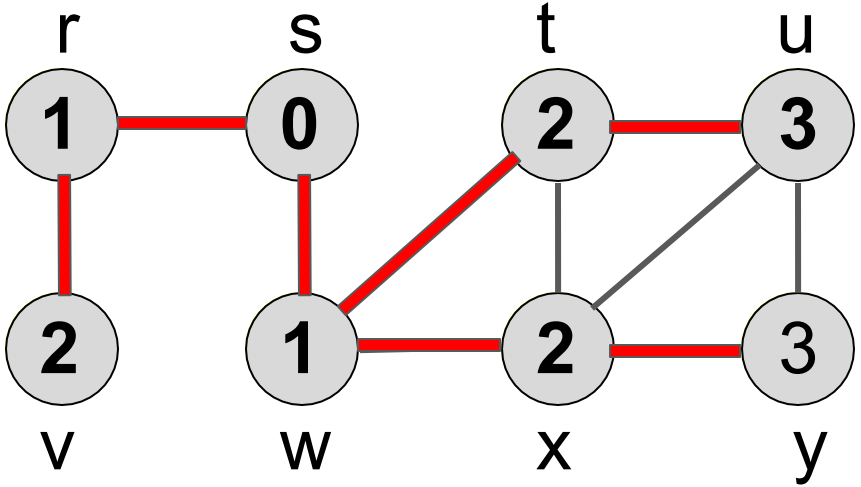


Queue

u	y
3	3

BFS Example

r	s	v	w	t	x	u	y
parents:	s	NULL	r	s	w	w	t	x



Queue

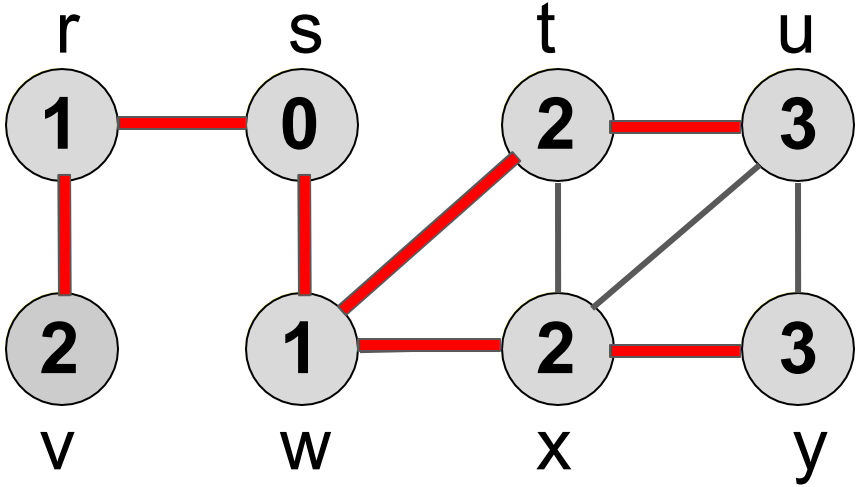
y

3

BFS Example

parents:

r	s	v	w	t	x	u	y
s	Null	r	s	w	w	t	x



Queue

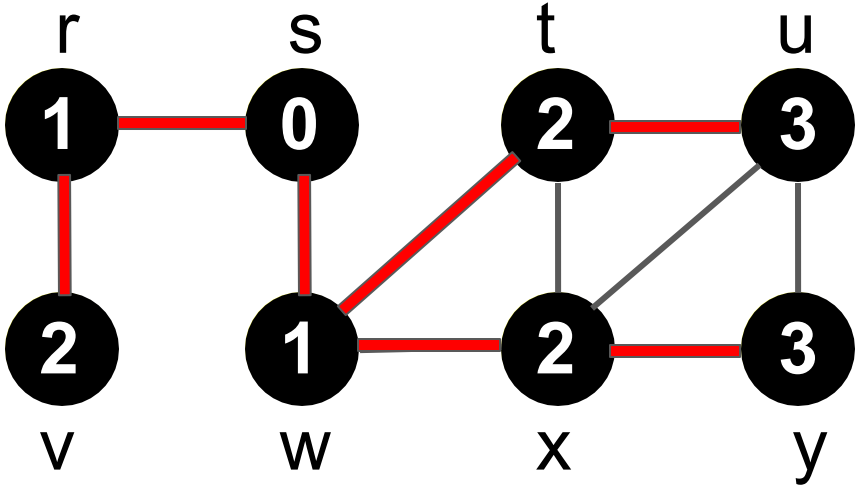
$\varnothing$

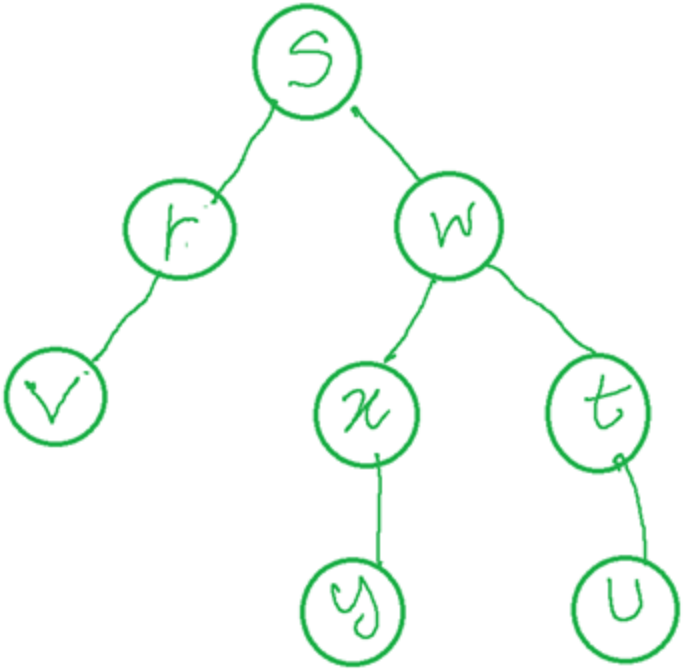
BFS: Tracking the path

```
BFS(G, s)
    Q = ∅
    for each node u in G:
        dist[u] = ∞
        pred[u] = NULL
    dist[s] = 0
    enqueue(Q, s)
    while Q not empty
        u = dequeue(Q)
        for each neighbor v of u:
            if dist[v] == ∞:
               dist[v]= dist[u] + 1
               pred[v] = u
               enqueue(Q, v)
```

Breadth-First Tree

- BFS builds a breadth-first tree:
  - A tree where the path from s to every node is the shortest path





BFS Application

- BFS can be used to check whether a graph is connected or not
  - Checking whether all vertices are marked discovered
- The connected component containing s
  - by returning all the vertices which are discovered
- Whether there is a path from s to v
  - Checking whether v is discovered

- **Exercises:**
  - Enhance BFS to find all connected components in time O(n + m)
    - If a graph is not connected
  - Use BFS to find if a connected graph has a cycle.

Breadth-First Tree

- Subgraph G'=(V', E') is generated after running the algorithm BFS
  - V' = {v ∈ V: pred[v] ≠ NULL} U {s}
    - V' consists of vertices in V which are reachable from s, since the algorithm sets pred[v]=u if and only if (u,v) is an edge in E and v is reachable from s → G' is a connected graph
  - E' = { (pred[v], v): v ∈ V' - {s} }
- G' is a Tree since it is connected and |E'| = |V'| - 1
- Since G' is a tree there is a unique simple path from s to every vertex in V'
- This path is the shortest path